

BRACKET · SECURITY REVIEW

Bracket Updates Review.

A security review of Updates Review.

CLIENT	PROJECT	AUDITORS	AUDIT WINDOW	FINAL REPORT
Bracket	Updates Review	Guardian	August 28 – September 18, 2025	September 24, 2025

§ 00

Contents.

49 pages — overview, confidence ranking, scope, 37 findings, disclaimer, and about Guardian.

§ 01	Overview	P. 03
§ 02	Confidence Ranking	P. 04
§ 03	Scope & Classifications	P. 05
	Files in Scope	P. 06
§ 04	Findings	P. 07
	Main Review · 17 findings	P. 10
	Remediation Review 1 · 16 findings	P. 27
	Remediation Review 2 · 2 findings	P. 44
	Remediation Review 3 · 2 findings	P. 46
§ 05	Disclaimer	P. 48
§ 06	About Guardian	P. 49

§ 01

Overview.

Guardian reviewed Updates Review across 4 review rounds. The final report includes 2 Critical, 8 High, 9 Medium, 14 Low, 4 Informational findings.

AUDIT FIRM	REPOSITORY
Guardian	bracket-fi/core-contracts
CLIENT	METHODOLOGY
Bracket	Smart contract · Manual · Main Review · Remediation Review 1 · Remediation Review 2 · Remediation Review 3
PROJECT	MAIN REVIEW COMMIT
Updates Review	dbc687a4fc86
PREPARED BY	REMEDATION REVIEW 1 COMMIT
Guardian	6dbd914dcd16
AUDIT WINDOW	REMEDATION REVIEW 2 COMMIT
August 28 – September 18, 2025	89185a954a23
FINAL REPORT	REMEDATION REVIEW 3 COMMIT
September 24, 2025	d8185fdcc63f

1.1 Confidence Ranking

2 / 5

Low Confidence

Guardian assigns a Confidence Ranking of 2 out of 5, combining finding severity, remediation outcome, and the reviewer’s assessment of latent risk. The full scale is on the next page.

1	2	3	4	5
VERY LOW CONFIDENCE	LOW CONFIDENCE	MODERATE CONFIDENCE	HIGH CONFIDENCE	VERY HIGH CONFIDENCE

1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
● Critical	2	0	0	0	2
● High	8	0	0	0	8
● Medium	9	0	7	0	2
● Low	14	0	8	0	6
● Informational	4	0	2	0	2
All severities	37	0	17	0	20

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence	Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity findings.
3 Moderate Confidence	Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.	Some High/Critical findings, with various Medium severity findings.
2 Low Confidence ASSIGNED	Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.	Over 6 High and Critical findings across the engagement.
1 Very Low Confidence	The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.	Over 12 High and Critical findings across the engagement.

§ 03

Scope & Classifications.

Guardian reviewed Updates Review in `bracket-fi/core-contracts`.

3.1 Severity Matrix

	IMP · HIGH	IMP · MED	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH

Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.

IMPACT · MEDIUM

Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.

IMPACT · LOW

Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.

LIKELIHOOD · HIGH

Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.

LIKELIHOOD · MEDIUM

Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.

LIKELIHOOD · LOW

Unlikely to ever occur in production.

Files in Scope.

FILE	NSLOC	TOTAL
No files were recorded in scope.		

§ 04

Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
MAIN REVIEW					
C-01	Shares Value Transferred	Logical Error	CRITICAL	RESOLVED	p. 10
C-02	Double Share Conversion Perturbs Accounting	Logical Error	CRITICAL	RESOLVED	p. 11
H-01	Missing Pause Functions	Logical Error	HIGH	RESOLVED	p. 12
H-02	Constructor Used For UUPS Contract	Logical Error	HIGH	RESOLVED	p. 13
H-03	Missing Ownership Initialization	Logical Error	HIGH	RESOLVED	p. 14
H-04	Balances Do Not Match Total Supply	Logical Error	HIGH	RESOLVED	p. 15
H-05	Incongruent Rates Used In Wrapping	Logical Error	HIGH	RESOLVED	p. 16
H-06	Upgrades Perturb Balances	Logical Error	HIGH	RESOLVED	p. 17
M-01	Nav Cannot Be Updated From Zero	Logical Error	MEDIUM	ACKNOWLEDGED	p. 18
M-02	Nav Updates May Be Unintentionally Constricted	Validation	MEDIUM	ACKNOWLEDGED	p. 19
L-01	Zero Wraps Allowed	Validation	LOW	RESOLVED	p. 20
L-02	Approval Mismatch	Unexpected Behavior	LOW	RESOLVED	p. 21
L-03	Misleading Conversion Functions	Warning	LOW	RESOLVED	p. 22
L-04	Missing Closed Case	Best Practices	LOW	RESOLVED	p. 23

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
L-05	Duplicated Active State	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 24
I-01	Missing Initializers	Best Practices	INFORMATIONAL	RESOLVED	p. 25
I-02	Dust Left In BracketWrappedVault.sol	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 26
REMEDATION REVIEW 1					
H-01	Vault Router Avoids KYC	Logical Error	HIGH	RESOLVED	p. 27
H-02	Incorrect assetsPerShare Starting Value	Logical Error	HIGH	RESOLVED	p. 28
M-01	Existing Allowances Change	Warning	MEDIUM	ACKNOWLEDGED	p. 29
M-02	Balances Do Not Match Total Supply	Logical Error	MEDIUM	RESOLVED	p. 30
M-03	KYCWhitelist Storage Corruption	Logical Error	MEDIUM	ACKNOWLEDGED	p. 31
M-04	Ineffective Blacklist	Logical Error	MEDIUM	RESOLVED	p. 32
M-05	Incorrect totalPendingShares Tracking	Logical Error	MEDIUM	ACKNOWLEDGED	p. 33
M-06	Whitelist Can Be Opened By Any Whitelisted User	Gaming	MEDIUM	ACKNOWLEDGED	p. 35
L-01	User Balances Stepwise Update	Warning	LOW	ACKNOWLEDGED	p. 36
L-02	Lock Frequency And Withdraw Delay Validation	Validation	LOW	RESOLVED	p. 37
L-03	No Way To Update VanityNav	Warning	LOW	ACKNOWLEDGED	p. 38
L-04	Unused Modifier	Superfluous Code	LOW	RESOLVED	p. 39
L-05	NavUpdater Deployment Warning	Warning	LOW	ACKNOWLEDGED	p. 40
L-06	Balances No Longer Include Inactive Amounts	Warning	LOW	ACKNOWLEDGED	p. 41
L-07	totalSupply Rounding	Rounding	LOW	ACKNOWLEDGED	p. 42

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
I-01	Unnecessary Import	Superfluous Code	INFORMATIONAL	RESOLVED	p. 43

REMEDIATION REVIEW 2

M-01	Whitelist Can Be Opened By Any Whitelisted User	Gaming	MEDIUM	ACKNOWLEDGED	p. 44
I-01	canTransfer Also Impacts Deposits And Withdrawals	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 45

REMEDIATION REVIEW 3

L-01	Withdrawals May Be Missed If Withdrawal Delay Is Updated	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 46
L-02	fixTotalNonMintedShares Can Be Called Again	Warning	LOW	ACKNOWLEDGED	p. 47

CRITICAL **C-01**

RESOLVED

Shares Value Transferred

CATEGORY Logical Error
LOCATION BracketWrappedVault.sol
REVIEW Main Review

DESCRIPTION

In the `unwrap` function the `transferFrom` invocation transfers the `amount` specified by the user, however this is a shares amount rather than a token amount.

RECOMMENDATION

Use the `assets` value in the `transferFrom` invocation instead.

RESOLUTION

Status: Resolved.

<https://github.com/bracket-fi/core-contracts/tree/b5becff1d66f121e3a0ccc12a82e9c0d90dcf826>

CRITICAL C-02

RESOLVED

Double Share Conversion Perturbs Accounting

CATEGORY Logical Error
LOCATION BracketVaultV2.sol
REVIEW Main Review

DESCRIPTION

The overridden `_update` function in the `RebasingToken` contract has been updated to include a conversion to shares for the amount being transferred. However the vault treats the amount being specified for the transfer/mint/burn action as a share amount.

Therefore in the `_clearDeposit` and `withdraw` functions when the `_mint` and `_burn` functions are called with values that have already been converted to shares, these values are doubly converted to shares thus perturbing the accounting system.

RECOMMENDATION

Consider removing the conversion to shares in the `_update` function, otherwise refactor the vault to treat the amounts passed into the transfer/mint/burn to be raw asset amounts rather than share amounts.

RESOLUTION

Status: Resolved.

<https://github.com/bracket-fi/core-contracts/tree/4c089d6108b10cd2ddfc01c58971b60fd835a9a0>

HIGH **H-01**

RESOLVED

Missing Pause Functions

CATEGORY	Logical Error
LOCATION	BracketWrappedVault.sol
REVIEW	Main Review

DESCRIPTION

The `BracketWrappedVault` contract inherits from `PausableUpgradeable` without implementing functions to expose the pause/unpause functionality.

As a result the wrap and unwrap functions cannot be paused.

RECOMMENDATION

Implement pause and unpaused permissioned external functions.

RESOLUTION

Status: Resolved.

<https://github.com/bracket-fi/core-contracts/tree/8e375447f20929a0a90e1253458d079026121344>

HIGH **H-02**

RESOLVED

Constructor Used For UUPS Contract

CATEGORY	Logical Error
LOCATION	BracketWrappedVault.sol
REVIEW	Main Review

DESCRIPTION

In the `BracketWrappedVault` contract the constructor is used to initialize the contract with the correct vault, decimals, name and symbol.

However the `BracketWrappedVault` contract is a UUPS implementation contract, therefore this assignment should occur in the initialize function to instantiate the storage slots of the proxy contract being used with this implementation contract. And the constructor of the implementation contract should use `disableInitializers` to disable the initialization of the UUPS implementation contract.

RECOMMENDATION

Move the constructor logic into an initialize function and use `disableInitializers` in the constructor of the `BracketWrappedVault` contract.

RESOLUTION

Status: Resolved.

<https://github.com/bracket-fi/core-contracts/tree/a0409cbcddb6f49d17666adc4145e98b8d9c2336>

HIGH **H-03**

RESOLVED

Missing Ownership Initialization

CATEGORY	Logical Error
LOCATION	BracketWrappedVault.sol
REVIEW	Main Review

DESCRIPTION

The `BracketWrappedVault` constructor fails to call the `__Ownable_init` function of the `OwnableUpgradeable` base contract. As a result no owner is assigned for the protocol and upgrades using the UUPS functionality cannot take place.

RECOMMENDATION

In the new initialize function for the `BracketWrappedVault` contract, invoke the `__Ownable_init` function and assign the appropriate owner.

RESOLUTION

Status: Resolved.

<https://github.com/bracket-fi/core-contracts/tree/a0409cbcddb6f49d17666adc4145e98b8d9c2336>

HIGH **H-04**

RESOLVED

Balances Do Not Match Total Supply

CATEGORY Logical Error
LOCATION RebasingToken.sol
REVIEW Main Review

DESCRIPTION

The `activeBalanceOf` has been changed to use the `vanityAssetsPerShare` function which maintains it's original behavior of fluctuating the current reported balance of users by the `vanityNav`, however the `activeSupply` function still uses the `assetsPerShare` function which is now reliant on the reported nav from the previous epoch.

As a result the reported values from the summation of user balances and the `totalSupply` result will not agree when the `vanityNav` does not match the reported nav from the previous epoch.

RECOMMENDATION

Update the `activeSupply` function to use the `vanityAssetsPerShare` function instead of the `assetsPerShare` function if it is desired that the supply and balances of users float with the `vanityNav`.

RESOLUTION

Status: Resolved.

<https://github.com/bracket-fi/core-contracts/tree/4c089d6108b10cd2ddfc01c58971b60fd835a9a0>

HIGH **H-05**

RESOLVED

Incongruent Rates Used In Wrapping

CATEGORY	Logical Error
LOCATION	BracketWrappedVault.sol
REVIEW	Main Review

DESCRIPTION

In the `BracketWrappedVault` the `convertToShares` and `convertToAssets` functions are used with the `assetsPerShare` rate to convert to and from shares.

However the `assetsPerShare` rate is not what will be used to determine how many shares ought to be transferred, instead the `vanityAssetsPerShare` rate is used in the `_update` function.

This will result in an unexpected amount of shares being transferred when the `vanityNav` does not match the `nav` of the last epoch.

RECOMMENDATION

Use the `vanityAssetsPerShare` instead of the `assetsPerShare` in the `wrap` and `unwrap` functions when converting between shares and assets.

RESOLUTION

Status: Resolved.

<https://github.com/bracket-fi/core-contracts/tree/4c089d6108b10cd2ddfc01c58971b60fd835a9a0>

HIGH **H-06**

RESOLVED

Upgrades Perturb Balances

CATEGORY	Logical Error
LOCATION	RebasingToken.sol
REVIEW	Main Review

DESCRIPTION

In the `_update` function a conversion to shares by the `vanityAssetsPerShare` rate has been introduced. This may be fine for new vaults which are being introduced, however for existing vaults this will immediately change the interpretation and valuation of entries in the balances mapping.

RECOMMENDATION

Do not perform this upgrade for vaults which already exist

RESOLUTION

Status: Resolved.

MEDIUM **M-01**

ACKNOWLEDGED

Nav Cannot Be Updated From Zero

CATEGORY Logical Error
LOCATION NavUpdater.sol
REVIEW Main Review

— DESCRIPTION

In the `_getDiff` function the `_currentNav` is always used in the denominator of the basis points diff calculation. As a result, if the nav ever goes to zero for any vault and is then updated to a nonzero value, the update will panic revert in the `_getDiff` function due to attempted division by 0.

— RECOMMENDATION

Consider early returning a default value of 0 from the `_getDiff` function to avoid division by zero in this case.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-02**

ACKNOWLEDGED

Nav Updates May Be Unintentionally Constricted

CATEGORY Validation
LOCATION NavUpdater.sol
REVIEW Main Review

— DESCRIPTION

The `maxBpsIncrease` value cannot be assigned higher than 10_000 basis points, meaning that any nav increase cannot reflect a doubling in the price of a bracket vault share.

This may inadvertently over-limit the appropriate amount of share value increase that can occur in a given epoch, since it could be reasonable for a fund to double it's nav in a week long epoch in certain outlier circumstances.

— RECOMMENDATION

Consider if the 10_000 basis point cap should hold for the increase validation, or if it should be higher.

— RESOLUTION

Status: Acknowledged.

LOW **L-01**

RESOLVED

Zero Wraps Allowed

CATEGORY	Validation
LOCATION	BracketWrappedVault.sol
REVIEW	Main Review

DESCRIPTION

In the `wrap` and `unwrap` functions, amounts of zero are allowed which may lead to unexpected behavior. To limit the unexpected paths that can be taken these values should be validated against.

In the `wrap` function, even non-zero but very small `amount` values can round to a zero `share` amount, so in the `wrap` function the `share` value should be validated to be nonzero.

RECOMMENDATION

In the `wrap` function, validate that the `share` value is not zero, and in the `unwrap` function validate that the `amount` value is not zero.

RESOLUTION

Status: Resolved.

<https://github.com/bracket-fi/core-contracts/tree/c870b1cdec566b4a230496550c2179459f791bb4>

LOW **L-02**

RESOLVED

Approval Mismatch

CATEGORY	Unexpected Behavior
LOCATION	RebasingToken.sol
REVIEW	Main Review

DESCRIPTION

The approval amounts in the rebasingToken contract are tracked and spent based on an asset basis rather than a share basis which leads to a mismatch between the shares which can be transferred relative to the approval amount over time.

For example, if User A approves User B to spend 10 amount at $T = 1$ when the assets per share is 1, then User B can spend 10 shares from User A.

However if at $T = 2$ the assets per share is 2, User B can now only spend 5 shares from User A.

This may simply be the expected behavior, since the value of the assets has gone up and so the approval is desired to be in the final amount rather than the shares amount.

However this creates notable behavior when rounding is taken into account, where small approval amounts can be spent without the transferring of actual assets. For example, if the asset per share rate is $1.1e18$, a user is approved to spend $10e18$ tokens from User A, and they transfer 1 wei of assets from User A the approval will decrease by 1 wei without transferring any assets from User A since the resulting transferred shares value rounds to zero.

RECOMMENDATION

Overall these behaviors are innocuous but worth noting and confirming if they are the expected behavior.

RESOLUTION

Status: Resolved.

<https://github.com/bracket-fi/core-contracts/tree/1afd8e338069105a7a93c9ab97bfece686be451a>

LOW **L-03**

RESOLVED

Misleading Conversion Functions

CATEGORY	Warning
LOCATION	RebasingToken.sol
REVIEW	Main Review

DESCRIPTION

In the `RebasingToken` contract the `convertToShares` and `convertToAssets` functions which accept only a shares amount have been introduced and are not used anywhere in the codebase. This indicates that they are for usage by integrating protocols or users interacting with the vault.

In this case, it should be noted that these functions may be misleading, since they use the `assetsPerShare` value which is based on the per share rate of the last epoch, which is not what is reflected in the user balances by way of the `vanityAssetsPerShare` rate being used in the `activeBalanceOf` function.

RECOMMENDATION

Be aware of this potentially misleading behavior of the `convertToShares` and `convertToAssets` functions.

RESOLUTION

Status: Resolved.

<https://github.com/bracket-fi/core-contracts/tree/4c089d6108b10cd2ddfc01c58971b60fd835a9a0>

LOW **L-04**

RESOLVED

Missing Closed Case

CATEGORY Best Practices
LOCATION KYCWhitelist.sol
REVIEW Main Review

DESCRIPTION

In the `canDeposit` and `canWithdraw` functions there is no case that handles the `CLOSED` `ActionState` nor is there a default return value for the function if none of the explicit cases are matched.

RECOMMENDATION

Consider adding a default `return false` statement to silence the compiler warning and implicitly cover the `CLOSED` `ActionState` case.

RESOLUTION

Status: Resolved.

<https://github.com/bracket-fi/core-contracts/tree/bb258c9ff5d92576fc2de9bc91e286418fc737ca>

LOW **L-05**

ACKNOWLEDGED

Duplicated Active State

CATEGORY	Unexpected Behavior
LOCATION	NavUpdater.sol
REVIEW	Main Review

— DESCRIPTION

In the `NavUpdater` contract there is an active state tracking for each vault, however this mirrors the active state tracking for each vault already done in the `KYCWhitelist` contract.

— RECOMMENDATION

Consider if this state can be deduplicated to simply rely on the `KYCWhitelist` tracking of the active state of a vault.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-01

RESOLVED

Missing Initializers

CATEGORY	Best Practices
LOCATION	BracketWrappedVault.sol
REVIEW	Main Review

DESCRIPTION

In the constructor for the `BracketWrappedVault` the initializer invocations are missing for the `PausableUpgradeable`, and `UUPSUpgradeable` contracts.

These initializer functions do nothing, however it is a best practice to call these initializer functions in the event that a future version of these base contracts in a future upgrade would have some logic.

RECOMMENDATION

Consider calling the initializer functions for the `PausableUpgradeable`, and `UUPSUpgradeable` contracts in the new `BracketWrappedVault` initializer function.

RESOLUTION

Status: Resolved.

<https://github.com/bracket-fi/core-contracts/commit/a0409cbcddb6f49d17666adc4145e98b8d9c2336>

INFORMATIONAL **I-02**

ACKNOWLEDGED

Dust Left In BracketWrappedVault.sol

CATEGORY Warning
LOCATION BracketWrappedVault.sol
REVIEW Main Review

— DESCRIPTION

Due to the round down conversion in the `convertToAssets` function there may be cases where a dust amount of shares are left held by the `BracketWrappedVault` contract after all users have withdrawn.

— RECOMMENDATION

Simply be aware of this behavior.

— RESOLUTION

Status: Acknowledged.

HIGH **H-01**

RESOLVED

Vault Router Avoids KYC

CATEGORY	Logical Error
LOCATION	VaultRouter.sol
REVIEW	Remediation Review 1

DESCRIPTION

The `VaultRouter` has been introduced to allow users to easily wrap WETH and deposit into the vault with. However this contract allows users to bypass the KYC restriction because the `BracketVault` performs validation on the `msg.sender` in the deposit function, which would be the `VaultRouter` contract, which must be whitelisted to function.

RECOMMENDATION

Either perform the whitelist validation also in the `VaultRouter` contract or consider removing the `VaultRouter` contract entirely.

RESOLUTION

Status: Resolved.

<https://github.com/bracket-fi/core-contracts/tree/89185a954a23a4bdc2db26a1b67cbe66113e96f9>

HIGH **H-02**

RESOLVED

Incorrect assetsPerShare Starting Value

CATEGORY Logical Error
LOCATION RebasingToken.sol
REVIEW Remediation Review 1

DESCRIPTION

The `BracketVault` has a decimals amount of whatever the underlying token has and uses `convertToShares` to determine the amount of vault tokens to mint.

However the default value for the `assetsPerShare` is $1e18$ when the epoch is 0, this leads to extreme precision loss and ultimately loss of funds for lower decimal vaults.

RECOMMENDATION

Report the default `assetsPerShare` as `10 ** _decimals`. Furthermore ensure that the updates for the nav for these vaults is in line with the decimals of the vault to avoid loss of funds and precision loss.

RESOLUTION

Status: Resolved.

<https://github.com/bracket-fi/core-contracts/tree/81962b087220567000f3fe0f751ec6d772fab43e>

MEDIUM **M-01**

ACKNOWLEDGED

Existing Allowances Change

CATEGORY Warning
LOCATION RebasingToken.sol
REVIEW Remediation Review 1

DESCRIPTION

Previously, the `transferFrom` and `transfer` functions accepted `value` parameters that represented `share` values. However now they have been updated to accept `value` parameters that represent `asset` values.

Originally, the approvals consumed in `transferFrom` would be in shares, however now the approvals consumed in `transferFrom` are based on the provided `value` parameter in assets.

So the net effect on approvals of this upgrade for existing vaults is that the approval values made by users will be changed from share values to asset values. Depending on the NAV of the vault this will either increase or decrease the amount transferrable by approvals for users.

RECOMMENDATION

Be aware of this change and ensure it will not cause any significant unexpected changes for users. Users should be aware of this change as well.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-02**

RESOLVED

Balances Do Not Match Total Supply

CATEGORY Logical Error
LOCATION RebasingToken.sol
REVIEW Remediation Review 1

DESCRIPTION

The `balanceOf` function has been updated to only rely on the `activeBalance` and ignore the `inactiveBalance`.

This is however contradictory to the implementation of the `totalSupply` function which reports the sum of the `activeSupply` and `inactiveSupply` as the `totalSupply`.

As a result the sum of the individual user balances does not match the aggregate `totalSupply` reported by the `BracketVault`.

RECOMMENDATION

If it is desired to only factor in the active shares, then update the `totalSupply` function to only track the `activeSupply`.

RESOLUTION

Status: Resolved.

<https://github.com/bracket-fi/core-contracts/tree/97c095171315fe52658a70fc8b755978225aabf8>

MEDIUM **M-03**

ACKNOWLEDGED

KYCWhitelist Storage Corruption

CATEGORY Logical Error
LOCATION KYCWhitelist.sol
REVIEW Remediation Review 1

DESCRIPTION

The `KYCWhitelist` contract is a `UUPSUpgradeable` implementation contract, however storage variables have been added before the `isUsedNonce` mapping which will change the storage values that it is mapped to.

This will reset all of the `usedNonce` reporting and allow replay of these signatures.

RECOMMENDATION

Only append new storage variables after the `isUsedNonce` mapping.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-04**

RESOLVED

Ineffective Blacklist

CATEGORY Logical Error
LOCATION Global
REVIEW Remediation Review 1

DESCRIPTION

The `KYCWhitelist` now has a blacklist mode, whereby only blacklisted addresses are considered not whitelisted for deposits and withdrawals.

However this blacklist is easily bypassed by a blacklisted user by transferring their deposit tokens or bracket vault tokens to a non-blacklisted address they control and performing the desired actions with the vault.

RECOMMENDATION

Be aware of this loophole and consider performing transfer checks if the KYC whitelist is in blacklist mode.

RESOLUTION

Status: Resolved.

<https://github.com/bracket-fi/core-contracts/tree/4ee243e1b43eb0ae531e6254e1d688a881ee63f4>

MEDIUM **M-05**

ACKNOWLEDGED

Incorrect totalPendingShares Tracking

CATEGORY	Logical Error
LOCATION	BracketVaultV2.sol
REVIEW	Remediation Review 1

DESCRIPTION

In the `BracketVault` contract the `totalPendingShares` variable is tracked to represent the total sum of all pending shares which technically belong to users but have not yet been minted to their underlying balances mapping entry.

The use case for this variable is to ensure that the `totalSupply` reported is the same as the sum of the user balances. However the tracking of the `totalPendingShares` does not correctly allow this.

The `totalPendingShares` variable is adjusted for withdrawals, however this has nothing to do with the pending shares that should be minted to users via deposits and thus perturbs the `totalSupply` accounting. For example, the `totalSupply` is based on the `totalMintedShares()` + `totalNonMintedShares()` result. `totalNonMintedShares` simply returns `totalPendingShares` for the `BracketVault` contract.

Whenever a withdrawal is initiated, it is both burned from the `totalMintedShares` and added to the `totalPendingShares`. This means there is no net change in the reported `totalSupply`, which does not agree with the underlying user balances tracking.

Furthermore, the `totalPendingShares` deduction by the `withdrawalShares` in the `_processDepositsWithdrawals` function is also invalid as this has no impact on the pending deposit shares of users.

RECOMMENDATION

Remove the deduction of `withdrawalShares` from the `totalPendingShares` in the `_processDepositsWithdrawals` function.

And remove the addition of `shares` to the `totalPendingShares` in the `withdraw` function.

RESOLUTION

Status: Acknowledged.

M-05 Incorrect totalPendingShares Tracking

CONTINUED

<https://github.com/bracket-fi/core-contracts/tree/48e0fb63acb9690a2c9b4deea71bbb5e98f3025c>

MEDIUM **M-06**

ACKNOWLEDGED

Whitelist Can Be Opened By Any Whitelisted User

CATEGORY Gaming
LOCATION Global
REVIEW Remediation Review 1

DESCRIPTION

With the advent of EIP 7702 accounts can now create code at their own address. This means that any whitelisted user can open up the whitelisted functionality to be used by non-whitelisted users.

Consider the following scenario:

- Assume protocol has a whitelist mapping, where only users who are whitelisted can call function A
- Bob is whitelisted for his address 0xFF
- Bob uses 7702 to set his 0xFF account code to:

```
contract {  
  address victimSystem;  
  function callThis(...) external {  
    victimSystem.whitelistedFunction(...);  
  }  
}
```

- Now anyone can call the whitelisted function through calling callThis on 0xFF

RECOMMENDATION

Be sure to monitor whitelisted accounts to ensure that they do not add functionality that opens up the whitelist to non-whitelisted accounts. Otherwise consider requiring that `msg.sender == tx.origin` for the whitelist.

RESOLUTION

Status: Acknowledged.

LOW **L-01**

ACKNOWLEDGED

User Balances Stepwise Update

CATEGORY Warning
LOCATION Global
REVIEW Remediation Review 1

DESCRIPTION

The `assetsPerShare` function has been updated to use the NAV value of the last epoch rather than the `vanityNav` of the current epoch.

As a result, for existing vaults this will cause a stepwise jump in the balance reported to users and the reported `totalSupply`.

This may cause unexpected issues for integrations which may rely on this data and may be unexpected for users.

RECOMMENDATION

Be sure to document this change for integrators and users who may be impacted.

RESOLUTION

Status: Acknowledged.

LOW **L-02**

RESOLVED

Lock Frequency And Withdraw Delay Validation

CATEGORY	Validation
LOCATION	BracketVaultV2.sol
REVIEW	Remediation Review 1

DESCRIPTION

The lock frequency was previously hardcoded at 7 days, however now it is configurable by the admin lite role and upon initialization.

The vault could however benefit from some validations on the frequency to ensure that it is neither configured to long or too short.

For example, the `BracketVault` contract will stop functioning after `type(uint16).max` epochs, therefore an epoch time of 10 minutes is too low.

Similarly, there is no validation that the configuration of the `withdrawDelay` is greater than some minimum and less than some maximum.

RECOMMENDATION

Consider if validations should be introduced which ensure the `lockFrequency` and `withdrawDelay` is bounded between a maximum and minimum range on initialization and updates.

RESOLUTION

Status: Resolved.

<https://github.com/bracket-fi/core-contracts/tree/4505682275d8551d05a4c1ba2e9efe4819d6679a>

LOW **L-03**

ACKNOWLEDGED

No Way To Update VanityNav

CATEGORY Warning
LOCATION BracketVaultV2.sol
REVIEW Remediation Review 1

DESCRIPTION

With the new `NavUpdater` contract there is no way to update the `vanityNav` with the `updateVanityNav` function as this function is not invoked in the `NavUpdater` contract.

RECOMMENDATION

Be aware of this unreachable function.

RESOLUTION

Status: Acknowledged.

LOW **L-04**

RESOLVED

Unused Modifier

CATEGORY	Superfluous Code
LOCATION	NavUpdater.sol
REVIEW	Remediation Review 1

DESCRIPTION

In the `NavUpdater` contract the `onlyActiveVault` modifier is unused.

RECOMMENDATION

Consider implementing it's use or removing it.

RESOLUTION

Status: Resolved.

<https://github.com/bracket-fi/core-contracts/tree/af232ad7e9fc74334065781b72680b0356e342e3>

LOW **L-05**

ACKNOWLEDGED

NavUpdater Deployment Warning

CATEGORY	Warning
LOCATION	NavUpdater.sol
REVIEW	Remediation Review 1

— DESCRIPTION

The `NavUpdater` has an open initialize function which can be called by anyone and used to grant the caller the default admin role of the contract.

If the NavUpdater has already received the `NAV_UPDATER_ROLE` for any vault instances a malicious actor could initialize it and unexpectedly start vaults or issue nav updates for existing vaults.

— RECOMMENDATION

Be sure to initialize the `NavUpdater` contract before it receives any permissions. Furthermore, ideally the `NavUpdater` contract is initialized in the same transaction as it's deployment, or at least in the same deployment script.

— RESOLUTION

Status: Acknowledged.

LOW **L-06**

ACKNOWLEDGED

Balances No Longer Include Inactive Amounts

CATEGORY Warning
LOCATION Global
REVIEW Remediation Review 1

DESCRIPTION

In the `RebasingToken` `balanceOf` function, the `inactiveBalanceOf` contributor has been removed, which will cause a stepwise change in the balances of users and Smart Contracts which have pending deposits.

This may be unexpected and could potentially cause issues in integrating systems.

RECOMMENDATION

Be sure to warn any integrators and users of this impending stepwise change.

RESOLUTION

Status: Acknowledged.

LOW **L-07**

ACKNOWLEDGED

totalSupply Rounding

CATEGORY	Rounding
LOCATION	Global
REVIEW	Remediation Review 1

DESCRIPTION

After the balances and totalSupply mismatch is resolved from M-02, `Balances Do Not Match Total Supply`, there remains a rounding error which invalidates the `sum of user balances == totalSupply` invariant.

This is due to rounding which occurs upon converting between users shares and asset amounts on transfers and deposit/withdrawals and on the calculation of user balances and totalSupply.

Extensive fuzz testing has shown that the inaccuracy can only occur in the direction of `sumOfUserBalances ≤ totalSupply`, and is at maximum off on the order of thousands of wei.

RECOMMENDATION

Be aware of this rounding difference and invalidation of the `sum of user balances == totalSupply` invariant.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-01**

RESOLVED

Unnecessary Import

CATEGORY Superfluous Code
LOCATION BracketWrappedVault.sol
REVIEW Remediation Review 1

DESCRIPTION

In the `BracketWrappedVault` contract the `IERC20` interface is explicitly imported but not used.

RECOMMENDATION

Consider removing the `IERC20` interface import.

RESOLUTION

Status: Resolved.

<https://github.com/bracket-fi/core-contracts/tree/1962de4dadbe27883ed820d920c5bf5e0107ea9>

MEDIUM **M-01**

ACKNOWLEDGED

Whitelist Can Be Opened By Any Whitelisted User

CATEGORY Gaming
LOCATION Global
REVIEW Remediation Review 2

DESCRIPTION

With the advent of EIP 7702 accounts can now create code at their own address. This means that any whitelisted user can open up the whitelisted functionality to be used by non-whitelisted users.

Consider the following scenario:

- Assume protocol has a whitelist mapping, where only users who are whitelisted can call function A
- Bob is whitelisted for his address 0xFF
- Bob uses 7702 to set his 0xFF account code to:

```
contract {  
  address victimSystem;  
  function callThis(...) external {  
    victimSystem.whitelistedFunction(...);  
  }  
}
```

- Now anyone can call the whitelisted function through calling callThis on 0xFF

RECOMMENDATION

Be sure to monitor whitelisted accounts to ensure that they do not add functionality that opens up the whitelist to non-whitelisted accounts. Otherwise consider requiring that `msg.sender == tx.origin` for the whitelist.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-01**

ACKNOWLEDGED

canTransfer Also Impacts Deposits And Withdrawals

CATEGORY Warning
LOCATION RebasingToken.sol
REVIEW Remediation Review 2

— DESCRIPTION

The `KYCWhitelist` `canTransfer` function has been added to the `_update` function and is now validated upon every invocation of `_update`. This means that the `canTransfer` validation applies for mints and burns when the from or to address is zero.

This may be unexpected given the naming of the predicate function, `canTransfer`, however seems appropriate given that this is how the blacklist is enforced.

— RECOMMENDATION

Consider if it is expected that the `canTransfer` validation is performed on mints and burns.

— RESOLUTION

Status: Acknowledged.

LOW **L-01**

ACKNOWLEDGED

Withdrawals May Be Missed If Withdrawal Delay Is Updated

CATEGORY Unexpected Behavior
LOCATION BracketVaultV2.sol
REVIEW Remediation Review 3

DESCRIPTION

In the `fixTotalNonMintedShares` function the `sumWithdrawals` value only increments until the `maxEpoch` which is assigned as `currentEpoch + withdrawDelay`.

However if the `withdrawDelay` is updated it may be possible that withdrawals exist past the defined `maxEpoch` and may be missed by the correction.

RECOMMENDATION

Be sure that the `maxEpoch` is in line with the latest withdrawal for each vault when the `fixTotalNonMintedShares` function is invoked.

RESOLUTION

Status: Acknowledged.

LOW **L-02**

ACKNOWLEDGED

fixTotalNonMintedShares Can Be Called Again

CATEGORY Warning
LOCATION BracketVaultV2.sol
REVIEW Remediation Review 3

DESCRIPTION

The `fixTotalNonMintedShares` function can be called again when the pending non minted shares return to zero. And for vaults which are never "fixed" this function can be called unintentionally.

RECOMMENDATION

This function is gated by the admin lite role so it is trusted. Simply be aware of this quirk.

RESOLUTION

Status: Acknowledged.

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

\$45B+DIGITAL ASSETS UNDER
REVIEW**200+**

ENGAGEMENTS COMPLETED

500+CRITICAL & HIGH FINDINGS
REPORTED

Reach us at audits@guardianaudits.com.